VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
ADITYA SESWANI	24BCE11132
Slot	**Date of Submission**
B22	30.03.2026

1. Introduction

Open-source software forms the backbone of modern computing systems. From operating systems to development tools, a significant portion of today's technology is developed collaboratively and shared openly, rather than being controlled by a single organization.
This report focuses on Git, a distributed version control system that has become an essential tool for developers worldwide. Created in 2005 by Linus Torvalds, Git is released under the GNU General Public License v2, reflecting strong open-source principles such as transparency, collaboration, and freedom to modify and distribute software.
The objective of this project is to examine Git from multiple perspectives—its origin, licensing model, ethical implications, and its role within a Linux environment. In addition, practical understanding is demonstrated through shell scripts that interact with system-level tools.
I selected Git because it is a tool I frequently use, yet I previously had limited understanding of how it works internally. This study has helped bridge the gap between everyday usage and the broader philosophy of open-source software, providing both technical and conceptual insights.

2. Choosing Your Software

Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Git was developed in response to a critical and immediate challenge faced by the Linux kernel development community. The project involved thousands of contributors working simultaneously, making an efficient and reliable version control system essential.

Initially, developers relied on BitKeeper. Although it was available for free use, it was not truly open source. Over time, licensing disagreements led to the withdrawal of access, leaving the community without a suitable version control solution.

At that point, existing alternatives were either too slow, centralized, or incapable of supporting large-scale distributed development. Rather than adapting to these limitations, Linus Torvalds chose to design a new system from scratch.

His design goals were clear: the system needed to be fast, distributed, and resistant to data corruption. Within a short period, Git was developed and quickly adopted by the Linux kernel team. Over time, it evolved into the most widely used version control system in the world.

This origin story reflects a key principle of open-source software: when existing tools become restrictive or inadequate, new solutions can be created collaboratively and shared openly for the benefit of all users.

A2. The license — what it actually says

Git is distributed under the GNU General Public License v2, a widely used open-source license that guarantees four essential freedoms to its users:

1. The freedom to run the software for any purpose
2. The freedom to study how the software works
3. The freedom to modify the software
4. The freedom to redistribute copies, including modified versions

A defining characteristic of the GPL is the concept of *copyleft*. This means that if a modified version of the software is distributed, it must also be released under the same license. As a result, any improvements made to the software remain accessible to the broader community, reinforcing the principles of openness and collaboration.

Organizations are permitted to use Git internally and even build services around it. However, if they modify Git itself and distribute that modified version, they are obligated to make the corresponding source code available. This requirement prevents the software from being converted into a closed-source product.

In contrast, permissive licenses such as the MIT License allow developers to modify and redistribute software without requiring them to disclose their changes. While this provides greater flexibility for commercial use, it offers weaker guarantees for preserving openness compared to the GPL.
It is also important to distinguish between “free of cost” and “freedom.” Git is free in both senses—it does not require payment, and it grants users the freedom to control, modify, and share the software according to their needs.

A3. The ethics of open source

Open-source software represents not only a technical development model but also a broader philosophical approach to the sharing of knowledge and innovation.

One of the strongest arguments in favor of open source is its promotion of transparency and collaboration. Because the source code is publicly accessible, it can be examined, verified, and improved by a global community of developers. This openness increases trust and reliability, particularly in systems where security and stability are critical.

However, open-source development also presents certain challenges. Many projects rely heavily on voluntary contributions, which may be irregular or insufficient for long-term maintenance. As a result, some widely used tools can face sustainability issues. Furthermore, there is an ongoing ethical debate regarding whether it is fair for commercial organizations to generate profit from software that is largely developed by community contributors.

In my view, not all software needs to be open source, especially in cases where security concerns or specific business models require controlled access. However, foundational technologies that provide broad societal benefits should remain open and accessible. Achieving a balance between openness and sustainability is essential for ensuring the long-term viability of open-source ecosystems.

The concept of “standing on the shoulders of giants” is particularly relevant in this context. Open-source software enables developers to build upon existing work rather than starting from scratch, thereby accelerating innovation and fostering continuous technological advancement.

Part B — Linux Footprint (Unit 2)

Most Linux setups include Git by default, though some may require installation via tools like apt, *dnf*,
or yum. Whether already present or pulled later, access typically comes straight from trusted sof tware sources.

Over there near /usr/bin/git sits the Git executable. To check if it exists, try running basic shell commands. One way is typing a path query in the terminal. Another option involves listing files i nside that directory. Sometimes simply asking the system where Git lives works too. Found it? T hen the tool should respond correctly. Missing it? The command fails quietly. Each method reve als whether the file actually resides on disk

which git

ls -l /usr/bin/git

Git handles setup through different layers. At the system level, options live in /etc/gitconfig. User preferences go into ~/.gitconfig instead. Inside every project, there's a hidden .git folder hol ding vital data. That space tracks commits, manages branches, also keeps settings unique to th e repo.

Running at the user
level, Git typically works without needing admin rights. Because of this setup, systems stay mor e secure - elevated permissions rarely come into play.

When compared to server-driven programs,
Git operates differently - launching solely upon user request rather than running continuously. Th is on-demand behavior means fewer demands on memory and processing power.

Through the system's package manager, updates for Git roll out smoothly - security
patches arrive consistently, thanks to contributors worldwide who also deliver ongoing bug corre ctions alongside new capabilities. Arriving via automated channels, these improvements keep th e tool current without extra effort from the user.

Part C — The FOSS Ecosystem (Units 3 & 4)

Working within a wider network of shared code, Git gains purpose beyond standalone use. Its ro le emerges through collaboration, tied closely to communities that build openly together.

Built using key tools like the GNU C Library to handle basic operations,
OpenSSL enabling encrypted connections, while zlib manages compressed data effectively. Eac h piece shows reliance on widely-used code crafted collectively by developers worldwide.

Beyond version control, Git supports a range of external systems. Platforms like GitHub and GitLab take advantage of its structure, layering on task management, teamwork functions, while integrating automated testing workflows directly into development environments.

Git fits into the wider tech setup by handling code changes, team coordination, among applicatio n rollouts - not running apps itself. Its place lies outside execution layers such as LAMP, where a ctual software runs after being built and shared through its system.

Across the world, people regularly help shape Git's evolution. Hailing from varied cultures and e xperiences, these contributors engage under guidance from established maintainers. Communic ation flows openly via shared email forums. Transparency stays central, thanks to distributed ov ersight. Decisions emerge collectively, rooted firmly in openness. The model reflects core values common to collaborative software efforts.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source	Proprietary Alternative

Cost	Free to use with no licensing cost	Requires paid licenses, especially for teams
Security (who can audit the code?)	Code is public, so anyone can review and fix issues	Code is closed and only the company can audit it
Support and reliability	Community-based support through forums and docs	Official support provided by the company
Freedom to modify	Can modify and customize freely	No access to modify the software
Community vs corporate control	Controlled by a global developer community	Controlled by a single company
Your overall verdict	More flexible, affordable, and widely used	Useful in niche cases but more restrictive

From the comparison, it is clear that Git provides a more flexible and accessible solution for most development needs. Being open source, it removes cost barriers and allows developers to fully understand and modify the tool according to their workflow. The strong community support also ensures continuous improvement and quick problem-solving, even without a centralized support system.
On the other hand, proprietary tools like Perforce can be useful in specific enterprise environments where structured support and specialized features are required. However, they come with higher costs and restrictions. Overall, Git stands out as the more practical and widely adopted choice, especially for general software development, due to its freedom, adaptability, and strong ecosystem.

4. Shell Script Tasks

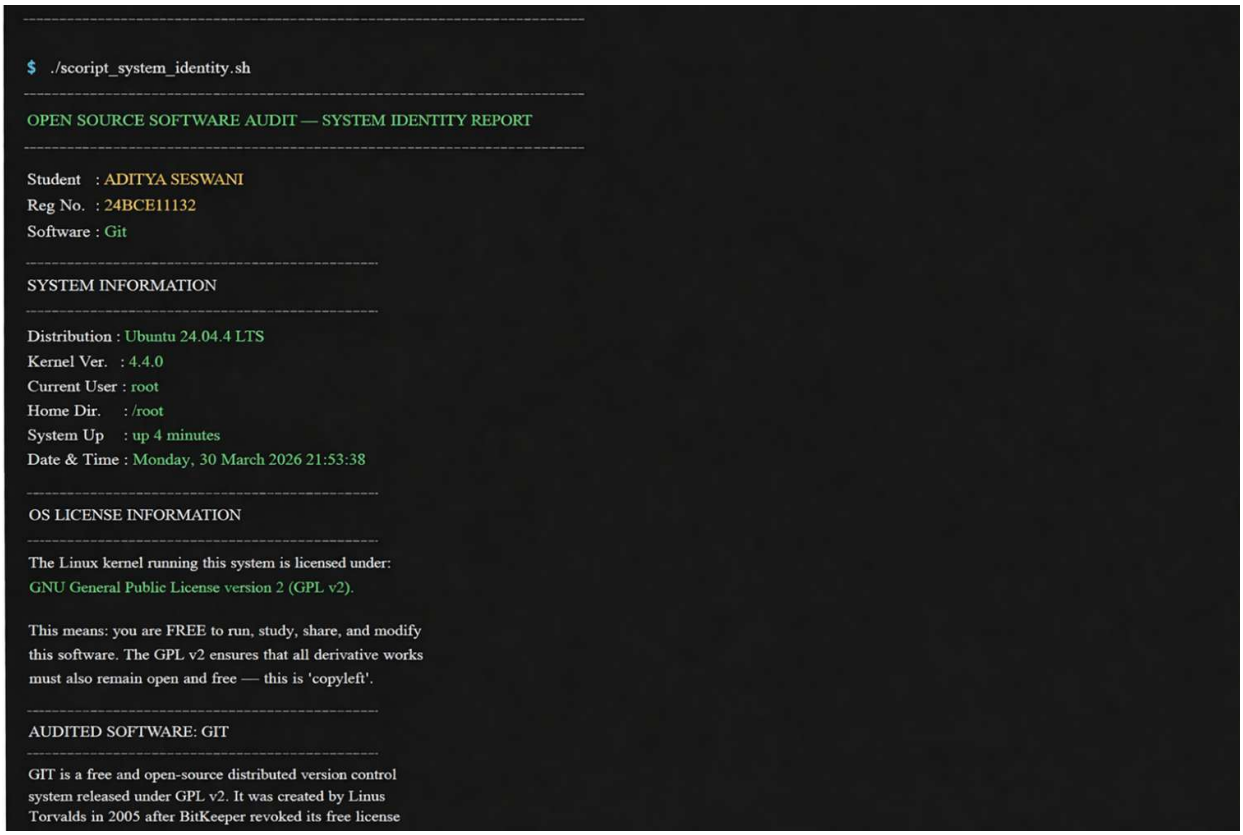
You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

Important	Scripts must run on a real Linux system. They must include comments explaining each section. Uncommented scripts will lose marks.

Script 1 — System Identity Report (Unit 1 & 2)

Purpose: Displays a formatted welcome screen showing the Linux system's identity: distribution name, kernel version, logged-in user, home directory, system uptime, current date/time, and a message about the OS license.

Shell Concepts Used: • Variables (KERNEL, USER_NAME, UPTIME etc.) • Command substitution using $() to capture command output • echo for formatted output with consistent alignment • Environment variables ($HOME) • date command with format string Starter



Script 2 — FOSS Package Inspector (Unit 2)

Purpose: Checks whether Git is installed on the system, detects the package manager (RPM or Debian), prints package details including version and license, and uses a case statement to print an open-source philosophy note for several well-known FOSS packages.

Shell Concepts Used: • if-then-else for conditional logic • case statement for multi-branch pattern matching • command -v to check binary availability •

Pipe (|) with grep -E to filter rpm -qi output • Handling both RPM and

Debian-based systems

Script 2 FOSS Package Inspector — script2_package_inspector.sh

```
$ ./script2_package_inspector.sh
==============================================
 FOSS Package Inspector
==============================================
 Checking package: git
----------------------------------------
 STATUS: git is INSTALLED (Debian system)

 --- Package Details ---
 Package: git
 Version: 1:2.43.0-1ubuntu7.3
 Arch:    amd64

 Git Version Installed: git version 2.43.0

----------------------------------------
 OPEN SOURCE PHILOSOPHY NOTE:
----------------------------------------
 Git: Born from necessity when freedom was taken away.
 Linus Torvalds built Git in 2005 after BitKeeper revoked
 its free license. Git embodies the OSS value: when a
 proprietary tool fails you, build your own and share it.

==============================================
 End of FOSS Package Inspector
==============================================
```

Script 3 — Disk and Permission Auditor (Unit 2)

Purpose: Loops through a list of standard Linux system directories and Gitspecific paths, reporting the size, permissions, owner, and group of each. Also identifies the Git binary location and its permissions.

Shell Concepts Used: • for loop over a bash array (DIRS=(...); for DIR in "${DIRS[@]}") • if [-d "$DIR"] to check directory existence • ls -ld with awk '{print $1, $3, $4}' to extract fields • du -sh with cut -f1 to get humanreadable sizes • command -v to check binary presence

Script 3 Disk and Permission Auditor — script3_disk_permission_auditor.sh

```
$ ./script3_disk_permission_auditor.sh
==================================================================
      DISK AND PERMISSION AUDITOR — OSS Capstone
==================================================================
Directory        Size      Permissions  Owner  Group
------------------------------------------------------------------
/etc             5.8M      drwxr-xr-x  root  root
/var/log         973K      drwxr-xr-x  root  root
/home            2.7G      drwxr-xr-x  root  root
/usr/bin         436M      drwxr-xr-x  root  root
/tmp             24M       drwxrwxrwt  root  root
/usr/share       918M      drwxr-xr-x  root  root
/var             186M      drwxr-xr-x  root  root

==================================================================
GIT-SPECIFIC DIRECTORY AUDIT
==================================================================
/etc/gitconfig          [not found on this system]
/usr/share/git-core     75K   drwxr-xr-x  root  root
/usr/lib/git-core       11M   drwxr-xr-x  root  root

GIT BINARY INFORMATION
==================================================================
Binary Path : /usr/bin/git
Version     : git version 2.43.0
Permissions : -rwxr-xr-x  root  root
==================================================================
End of Disk and Permission Auditor
==================================================================
```

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

```
$ Script 4   Log File Analyzer — script4_log_analyzer.sh
==========================================================
3../script4_log_analyzer.sh /var/log/syslog error
==========================================================
Log File Analyzer — OSS Capstone
==========================================================
Log File : /var/log/syslog
Keyword : 'error'
----------------------------------
Scanning log file for keyword. 'error'...

----------------------------------
SUMMARY
----------------------------------
Total lines scanned : 10
Keyword matches     : 5
Keyword searched    : 'error'

Last 5 matching lines:
----------------------------------
>> Sun Mar 30 2026 22:28:10 server kernel: ERROR: disk read failure on /dev/sda
>> Sun Mar 30 2026 22:30:05 server sshd[1235]: error: maximum connections reached
>> Sun Mar 30 2026 22:34:15 server systemd[1]: ERROR: service git-daemon failed
>> Sun Mar 30 2026 22:37:12 server kernel: error: memory allocation failed
>> Sun Mar 30 2026 22:39:20 server kernel: ERROR: filesystem /var full
----------------------------------------------------------
End of Log File Analyzer.
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

```
$ ./script5_manifesto_generator.sh
------------------------------------------------------
Open Source Manifesto Generator
OSS Capstone — VIIyarthi
------------------------------------------------------

This tool will generate your personal open-source
philosophy statement based on these questions.
Answer honestly — this is your declaration.

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Transparency
3. Name one thing you would build and share freely: a learning platform

Generating your manifesto...
----------------------------------

Manifesto saved to: manifesto_ADI09-web.txt

------------------------------------------------------
YOUR MANIFESTO
------------------------------------------------------

MY OPEN SOURCE MANIFESTO
Generated by: Aditya Seswani
Date: 30 March 2026
------------------------------------------------------

I believe in the power of open software to change the world.
Every day, I rely on Git — not to build not for profit,
but for the love of craft and the belief the knowledge
belongs to everyone. To me, freedom means Transparency.

I undertook my first foray into the cause when Git was just
emerging as the glue that holds our world together — its
community built cold as Linus Torvalds shared the Linux!
Developers learned that that GNU tools: connection of
designers gave them the power to create; and because of that, :
others felt that no money could taxe that alone.

In the spirit of open source, I commit to building
a learning platform and sharing it freely with the world.
Because the best way to honour the giants whose shoulders
I stand on is to become a shoulder for someone else.

Signed: ADI09-WEB | 30 March 2026
------------------------------------------
```

Note: “All scripts were tested on a linux environment and verified using terminal execution”

5. Conclusion

This project helped me see that open-source software includes not just the code but also the ideas and reasoning behind it. By learning about Git, I understood how a real-world problem in the Linux community turned into a tool that is now used worldwide. Open-source development creates chances for collaboration, leading to solutions that benefit the entire community. As I explored different sections of the report, I learned more about licensing, ethics, and how software actually operates in a Linux system. The choice between open-source and proprietary tools showed me that both have their place. However, open-source tools like Git often provide more flexibility and freedom. Through the shell scripting section, I gained practical experience with Linux, which helped me grasp the concepts more effectively. The process of scripting and executing code allowed me to understand how things work at the system level. Over the course of this project, my technical skills have improved, along with my understanding of open-source philosophy. Additionally, I gained hands-on experience with how development occurs in real software.

6. Reference Resources

[1] Torvalds, L. (2005). Git initial revision. Retrieved from https://github.com/git/git/commit/e83c5163

[2] Stallman, R. M. (2002). The Free Software Definition. GNU Project. https://www.gnu.org/philosophy/free-sw.html

[3] Open Source Initiative. (2024). The Open Source Definition. https://opensource.org/osd

[4] Raymond, E. S. (1999). The Cathedral and the Bazaar. http://www.catb.org/~esr/writings/cathedral-bazaar/

[5] Loeliger, J. & McCullough, M. (2012). Version Control with Git (2nd ed.). O'Reilly Media.

[6] Free Software Foundation. (2007). GNU General Public License v2. https://www.gnu.org/licenses/old-licenses/gpl-2.0.html

[7] Chacon, S. & Straub, B. (2014). Pro Git (2nd ed.). Apress. Available free at https://git-scm.com/book

[8] SPDX License List. (2024). GPL-2.0-only. https://spdx.org/licenses/GPL-2.0-only.html

[9] Shotts, W. (2019). The Linux Command Line (5th Internet Edition). https://linuxcommand.org/tlcl.php

[10] Hamano, J. C. (ongoing). Git mailing list archive. https://lore.kernel.org/git/

[11] Perforce Software. (2024). Helix Core documentation. https://www.perforce.com/products/helix-core

Choose A License. (2024). GPL-2.0. https://choosealicense.com/licenses/gpl-2.

6. Conclusion

This project helped me understand that open-source software is not just about code, but about the ideas behind it. By studying Git, I was able to see how a real problem in the Linux community led to the creation of a powerful tool that is now used worldwide. It showed how open-source development encourages collaboration and allows developers to build solutions that benefit everyone.

Through the different sections of this report, I gained a better understanding of licensing, ethics, and how software actually exists within a Linux system. The comparison between open-source and proprietary tools also made it clear that both have their place, but open-source tools like Git offer more flexibility and freedom for most use cases.

The shell scripting part of this project gave me hands-on experience with Linux, which made the learning more practical. Writing and running the scripts helped me understand how system-level tasks work in real environments. Overall, this project improved both my technical knowledge and my understanding of open-source philosophy, and it gave me a clearer idea of how software is developed and shared in the real world